CS 2143 · OBJECT-ORIENTED PROGRAMMING

Test 1 — C++ Review & OOP Basics

50-minute exam — draws from Quizzes 01–03 & Worksheets 01–04

Name _____ Date _____ Section _____

part 1 Multiple Choice

circle the best answer — 1 pt each

1. By default, members of a class declared with the `struct` keyword are:

Ⓐ private Ⓑ public Ⓒ protected Ⓓ decided by the compiler

2. An array-based `Stack` only needs one index, `top`, because pushes and pops always happen:

Ⓐ at the same end Ⓑ at opposite ends Ⓒ at a random index Ⓓ only during construction

3. A linked-list `Queue` needs *two* pointers, `front` and `rear`, because it:

Ⓐ adds and removes at opposite ends Ⓑ stores twice the data Ⓒ must be sorted

Ⓓ is required by every linked list

4. Given `Widget a(5);` then `Widget b(a);` and `Widget c = a;`, both `b` and `c` are built by the:

Ⓐ assignment operator Ⓑ copy constructor Ⓒ default constructor Ⓓ destructor

5. `operator<<` normally has to be written as a non-member (`friend`) function because:

Ⓐ the left operand (`cout`) isn't your class Ⓑ member functions can't return references

Ⓒ `friend` is required for any operator Ⓓ it runs faster as a free function

6. A class owns a raw pointer (`int* data;`) allocated with `new`. Which three members form the “Rule of Three”?

Ⓐ constructor, destructor, `operator<<` Ⓑ copy constructor, destructor, `operator=`

Ⓒ default constructor, copy constructor, getter Ⓓ destructor, `operator+`, comparison operator

Test 1 — C++ Review & OOP Basics

50-minute exam — draws from Quizzes 01–03 & Worksheets 01–04

part 2 Match the Concept

write the letter — 1 pt each

Match each term on the left to its best definition below.

Encapsulation _____

Inheritance _____

Polymorphism _____

A. Overloading or overriding methods so behavior adapts to the context it's used in.

B. Bundling data with the functions that operate on it, and hiding the internal details from outside code.

C. Organizing classes hierarchically so classes lower in the hierarchy reuse and extend behavior from classes above.

Test 1 — C++ Review & OOP Basics

50-minute exam — draws from Quizzes 01–03 & Worksheets 01–04

part 3 Find the Bug

4 pts

COMPILES? OR NOT? ACCOUNT.CPP

```
class Account {
    Account(double balance) {
        this→balance = balance;
    }
    double balance;
public:
    double getBalance() { return balance; }
};

int main() {
    Account a(500.00);
}
```

Circle one: **compiles** / **does not compile**. If it does not compile, which line fails and *why*? If it does compile, what would need to change for `a.balance` to be readable from `main()`?

Test 1 — C++ Review & OOP Basics

50-minute exam — draws from Quizzes 01–03 & Worksheets 01–04

part 4 From Usage to Class

the big one — show your reasoning — 10 pts

Below is real usage of a `Song` class — no definition has been given to you. Just like every "class snippet" question in the study guide, work backward from the calls to figure out what the class must look like.

USAGE — MAIN.CPP

```
1   Song S0;
2   Song S1("Bohemian Rhapsody", "Queen", 355);
3   Song S2("Imagine", "John Lennon");
4   S1.setGenre("Rock");
5   S2.play();
```

In the box below: **(a)** list every data member you can identify, with its data type — **(b)** if something is ambiguous (like lines 2 & 3 using different constructors), don't guess — leave a comment saying you noticed — **(c)** write the full class definition: constructors, getters/setters for every data member, and a declaration (not implementation) for any method the usage calls but you weren't given.

```
class Song {
```

```
};
```

Test 1 — C++ Review & OOP Basics

50-minute exam — draws from Quizzes 01–03 & Worksheets 01–04

part 5 Implement the Methods

scope resolution — 6 pts

Given the class below, implement `Increment` and `Reset` as if writing them *outside* the class definition. `Increment` adds the incoming value to `count`; `Reset` sets `count` back to `0`.

```
class Counter {
private:
    int count;
public:
    Counter();
    void Increment(int);
    void Reset();
};
```

Test 1 — C++ Review & OOP Basics

50-minute exam — draws from Quizzes 01–03 & Worksheets 01–04

part 6 From Usage to Class, Round Two

show your reasoning — 8 pts

Same drill as Part 4, different class. Work backward from the usage below to figure out `Employee`.

USAGE — MAIN.CPP

```
1   Employee E0;
2   Employee E1("Maria Chen", "Engineer", 82000);
3   Employee E2("Alex Ortiz", "Intern");
4   E1.giveRaise(5000);
5   E2.clockIn();
```

In the box below: **(a)** list every data member you can identify, with its data type — **(b)** flag anything ambiguous with a comment instead of guessing — **(c)** write the full class definition: constructors, getters/setters for every data member, and a declaration for any method the usage calls but you weren't given.

```
class Employee {
```

```
};
```